# Phase ②.5 Safety-Guided Diffusion 设计 (research output)

## 0. TL;DR

For an iDP3-based UAV obstacle-avoidance policy with truncated DDIM (t_start=10, 5 inference steps), CFG (λ_cfg=1.5), and anchor-head initialization, **the right safety-guidance recipe is gradient guidance on the predicted clean trajectory $\hat{x}_0$ (Tweedie estimate), using a softplus-hinge k-NN distance to the body-frame point cloud, injected on the CFG-combined noise prediction ε_cfg with a cosine-shaped per-step scale α_j that peaks in the middle of the 5-step window**. This is the Lan-o3dp/MPD-style "soft-constraint via classifier guidance" pattern, not a SafeDiffuser-style CBF-QP (which we cannot afford at 24 GB/T4 and 5 denoising steps, and which would require per-step QP solves that risk breaking anchor and CFG behavior). The hard-constraint MC-1 (no post-hoc) is honored because the guidance is embedded inside the denoising loop, not appended after. Expected inference-time overhead: 130–180 ms per plan (well within the 500 ms slack of the 3 s budget).

Three bullets:

- **Math form**: Use $\varepsilon_{safe} = \varepsilon_{cfg} + \sqrt{(1-\bar{\alpha}_j)}\cdot s_{safe}\cdot\alpha_j\cdot\nabla_{x_j} c(\hat{x}_0(x_j))$, where c is a softplus-hinge k-NN penalty evaluated on the cumsum'd world-frame waypoints derived from the predicted clean delta-action trajectory. This is the Lan-o3dp / DPS / MPD pattern adapted to ε-prediction.
- **Compatibility**: Add guidance *after* CFG combination on ε (not on x_{t-1}), keep CFG λ_cfg=1.5 unchanged, leave anchor initialization untouched, and clip the per-waypoint gradient by L2-norm 1.0 to avoid overriding the anchor's "direction choice".
- **Numerics to start**: d_safe = 0.35 m (matches the 0.30 m EGO B-spline inflation + 0.05 m margin), s_safe = 2.0 (then sweep 0.5–5.0), guidance schedule $\alpha_j = \cos^2(\pi\cdot(j-2)/4)$ so it is strongest in the middle 3 of 5 steps, gradient-norm clip = 1.0, k = 8 nearest points, softplus β = 20.

---

## 1. 文献综述 (8 篇核心论文, verbatim 引用)

### 1.1 SafeDiffuser (Wei Xiao, Tsun-Hsuan Wang, Chuang Gan, Ramin Hasani, Mathias Lechner, Daniela Rus — ICLR 2025, arXiv:2306.00148; proceedings.iclr.cc paper hash f95606d8e870020085990d9650b4f2a1)

Verbatim, ICLR 2025 proceedings, Definition 3.3:

> "A denoising diffusion procedure (1) is defined to be a SafeDiffuser if the corresponding diffusion dynamics (6) satisfy the following certificate: $$\frac{db(\tau^j_k)}{d\tau^j_k}\nu^j_k + h_{k,1}(j) + \alpha(b(\tau^j_k) - h_{k,2}(j)) \ge 0,\quad \forall k\in\{0,\dots,H\},\ \forall j\in\{0,\dots,N-1\}$$" (iclr) (iclr)

The Robust-Safe variant (Theorem 3.4) sets:

> "$h(\nu_k^j|\tau_k^j) = \frac{db(\tau_k^j)}{d\tau_k^j}\nu_k^j + \varepsilon(b(\tau_k^j) - \gamma(N,\varepsilon)), \varepsilon > 0$" (iclr)

Enforcement is a per-step minimum-deviation QP (Eq. 13):

> "$\nu^{j*} = \arg\min_{\nu^j} ||\nu^j - \frac{\tau^j - \tau^{j+1}}{\Delta\tau}||^2$, s.t. (8) if RoS diffuser else s.t. (12)" (iclr)

Key reported computation times (Table 2, maze):

> "ROS-DIFFUSER (OURS) ... TIME 0.106 ... RES-DIFFUSER-L10 (OURS) ... TIME 0.011" (iclr)

i.e., **applying invariance over only the last 10 diffusion steps drops per-step cost from ~106 ms to ~11 ms** — important for our 5-step truncated window.

### 1.2 CGD: Constraint-Guided Diffusion (Kota Kondo, Andrea Tagliabue, Xiaoyi Cai, Claudius Tewari, Olivia Garcia, Marcos Espitia-Alvarez, Jonathan P. How — MIT Aerospace Controls Laboratory, arXiv:2405.01758, 2024)

Most directly UAV-relevant. They split into block-coordinate sub-problems and use a *push-out* gradient guidance for collision avoidance (Eq. 6, verbatim): (arXiv)

> "$q_{pos} \leftarrow q_{pos} + \frac{\alpha_{\text{obst}}}{\|q_{pos} - q_{\text{obst}}\|} q_{\text{dir}}$" (arxiv)

where $q_{\text{dir}} = \frac{q_{pos} - q_{\text{obst}}}{\|q_{pos} - q_{\text{obst}}\|}$ and they explicitly set:

> "$\alpha_{\text{obst}}$ is a constant pushing scale, and in our case set to 1.0." (arxiv)

CGD also uses a QP for dynamic feasibility (Eq. 3):

> "$\min_q \frac{1}{2}\|q_{NN} - q\|_Q^2$ s.t. $A_{\text{dyn}} q \leq b_{\text{dyn}}$" (arxiv)

Critical for us: CGD treats obstacles as **explicit centers** $q_{\text{obst}}$, which we do *not* have — we have a 4096-point cloud and no segmentation. Their pushing form is therefore not directly transferable; we replace it with a k-NN soft-min.

### 1.3 Diffuser (Michael Janner, Yilun Du, Joshua B. Tenenbaum, Sergey Levine — ICML 2022, arXiv:2205.09991)

The foundational classifier-guidance form for trajectory diffusion. From Dhariwal & Nichol (2021), adopted by Janner et al.:

> "$\mu_t' = \mu_t + s\,\Sigma_t(x_t)\,\nabla_{x_t} \log p_{\text{cl}}(y|x_t)$" :antCitation[]{citations="442a4c3e-d6fa-4639-a62b-e249202173da" injected="space"}

Janner et al. use this with reward gradient at every reverse step. Our adaptation: replace $\log p_{\text{cl}}(y|x_t)$ with $-c_{\text{safe}}(x_t)$, a collision cost. (Diffusion-planning)

### 1.4 Lan-o3dp (Hang Li, Qian Feng, Zhi Zheng, Jianxiang Feng, Zhaopeng Chen, Alois Knoll — "Language-Guided Object-Centric Diffusion Policy for Generalizable and Collision-Aware

**Robotic Manipulation", arXiv:2407.00451 v3, updated 16 Mar 2025, accepted ICRA 2025)**

Directly relevant: a **training-free, inference-time cost-guided trajectory generation** for collision avoidance on a 3D-diffusion-policy backbone. The key innovation: compute the cost gradient on the **estimated clean trajectory** $A_{0|k}$ rather than on the noisy $x_t$: (arXiv + 2)

> "if $D(A_{0|k}, C_{\text{ob}})$ is shorter than a safety critical distance $Q^*$, a non-zero gradient will be assigned to the corresponding waypoint." (arXiv)

> "Sample vs. epsilon prediction: Although the 'sample' prediction shows a bit higher success rate than 'epsilon' prediction in the simulation, we found that the guidance is almost ineffective if each step predicts the initial sample $A_0$ directly. Using sample prediction can not successfully avoid obstacles. Since there is not much performance drop of our proposed method when using epsilon prediction, we use epsilon prediction in real-world." (arXiv)

This is **exactly our setting**: ε-prediction, point-cloud obstacles, gradient on the Tweedie estimate of $x_0$. [UNCERTAIN: the exact numerical value of $Q^*$ and the guidance scale γ are in the implementation-details appendix of v3 and were not retrievable in this research session due to arxiv rate limits; the gradient scale we recommend (s_safe=2.0) is derived from MPD's range and Lan-o3dp's qualitative report rather than a verbatim Lan-o3dp number.]

### 1.5 Motion Planning Diffusion (João Carvalho, An T. Le, Mark Baierl, Dorothea Koert, Jan Peters — IROS 2023, arXiv:2308.01557; extended journal version arXiv:2412.19948)

Verbatim guidance form (Section III-D):

> "$g = \nabla_{\tau_{t-1}} \log p(O|\tau_{t-1}) = -\sum_i \lambda_i \nabla_{\tau_{t-1}} c_i(\tau_{t-1})$" (tu-darmstadt)

Per-step update (Algorithm 1, line 11):

> "$\tau_{t-1} = \mu_t + g + \sigma_t z,\ z \sim \mathcal{N}(0, I)$" (tu-darmstadt)

Collision cost (hinge on signed distance):

> "$c(x) = \begin{cases} -d(x) + \varepsilon & \text{if } d(x) \le \varepsilon \\ 0 & \text{if } d(x) > \varepsilon \end{cases}$" (tu-darmstadt)

Crucial practical note:

> "In practice, to keep the influence of the task likelihood, we typically drop the covariance $\Sigma_t$ scaling … This choice is equivalent to scaling $\lambda_i = \tilde{\lambda}_i / \sigma_t$ with time-dependent variances $\sigma_t$, where $\tilde{\lambda}_i$ are the constant hyperparameters, which can be interpreted as increasing likelihood relevancy when $\sigma_t \to 0$." (tu-darmstadt)

This **directly motivates an annealed schedule that grows as denoising progresses** — opposite of image-domain CFG schedules but consistent with planning literature. MPD's diffusion schedule (Sec. IV-A): "The diffusion models are trained for 25 diffusion steps with exponential scheduling"; guidance applied at every reverse step. (tu-darmstadt)

**1.6 DiffusionDrive (Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, Xinggang Wang — CVPR 2025 Highlight, pp. 12037–12047, doi:10.1109/CVPR52734.2025.01124; HUST & Horizon Robotics; arXiv:2411.15139)**

Confirms the **truncated diffusion + anchor** recipe we use:

> "We stack 2 cascade diffusion decoder layers and apply truncated diffusion policy with 20 clustered anchors. The training diffusion schedule is truncated by 50/1000 to diffuse the anchors, while during inference, we use only 2 denoising steps." (arxiv)

Forward truncated process (Eq. 4):

> "$\tau_k^i = \sqrt{\bar{\alpha}^i} a_k + \sqrt{1-\bar{\alpha}^i}\varepsilon$, where $i \in [1, T_{\text{trunc}}]$ and $T_{\text{trunc}} \ll T$ is the truncated diffusion steps." (arxiv)

DiffusionDrive does **not** add any safety guidance on top of truncation — safety is implicit via the imitation + anchor classification loss (Eq. 6: "$L = \sum_k [y_k L_{rec}(\hat{\tau}_k, \tau_{gt}) + \lambda \text{BCE}(\hat{s}_k, y_k)]$"). Our Phase ②.5 fills exactly that gap. Their ablation on number of denoising steps (Table 4): Steps=1 → PDMS 87.9; Steps=2 → 88.1; Steps=3 → 88.1 — i.e., almost flat past step 2, confirming that very few steps suffice when anchors carry the structural prior. (arxiv)

**1.7 CAPE — Context-Aware Diffusion Policy via Proximal Mode Expansion (Rui Heng Yang, Xuan Zhao, Leo Maxime Brunswic, Montgomery Alban et al., corresponding: Amir Rasouli — arXiv:2511.22773, submitted 27 Nov 2025)**

Verbatim from the Introduction, confirming the trade-off framing:

> "this approach involves a brittle trade-off: weak guidance may be insufficient to prevent unsafe trajectories, while strong signals risk distorting the learnt distribution, leading to degraded performance and unrealistic trajectories." (arxiv)

This motivates our **per-step annealed scale + gradient-norm clip** to keep the anchor's mode choice intact.

**1.8 PC-Diffuser — Path-Consistent Capsule CBF Safety Filtering (Eugene Ku, Yiwei Lyu, arXiv:2603.10330, 2025)**

Provides the comparative framing — three families: Classifier Guidance, SafeDiffuser, MPC-CBF:

> "Classifier Guidance steers the denoising process via gradients of a hand-crafted energy-based classifier aiming to increase the distance from vehicle in collision. SafeDiffuser incorporates barrier constraints on the denoising process … MPC-CBF embeds CBF constraints into the receding-horizon optimization." (arXiv)

The paper reports reducing collision rate from 100% to 10.29% on the nuPlan all-collision challenge set (Val14 subset), outperforming Classifier Guidance, SafeDiffuser, and MPC-CBF baselines — a useful upper-bound reference for the regime we operate in. (arXiv)

We will adopt the **Classifier-Guidance family** (proven cheap and compatible with CFG + truncation), not the CBF-QP family (too costly at 5 steps with our delta-action representation, and the per-step QP would need a full Jacobian through the cumsum + softmin chain, conflicting with anchor stability).

---

## 2. 数学完整推导

### 2.1 Notation reconciliation with iDP3

Let:

- $x_j \in \mathbb{R}^{H \times d_a}$ be the noisy action trajectory at DDIM step $j \in \{j_{\text{start}} = 10, 8, 6, 4, 2, 0\}$ (5 inference steps over an originally 50-step schedule, ratio 0.2). $H = 16, d_a = 11$.
- $\varepsilon_\theta(x_j, j, o_{\text{cond}})$ = noise-prediction U-Net with conditioning $o_{\text{cond}}$ (4096-pt body-frame point cloud feature + 9-d state + anchor embedding).
- $\bar{\alpha}_j$ = cumulative noise schedule (DDPM convention).
- $\hat{x}_0(x_j) := \frac{x_j - \sqrt{1-\bar{\alpha}_j}\,\varepsilon_\theta(x_j, \cdot)}{\sqrt{\bar{\alpha}_j}}$ = Tweedie / "predicted $x_0$" estimate (the same expression used in iDP3's DDIM scheduler step, e.g. mmagic line `pred_original_sample = (sample - (beta_prod_t)**0.5 * model_output) / alpha_prod_t**0.5`). Readthedocs

### 2.2 Existing CFG step in iDP3 (verbatim form)

iDP3 uses standard classifier-free guidance over the noise prediction:

$$\varepsilon_{\text{cfg}}(x_j) = \varepsilon_\theta(x_j, \varnothing) + \lambda_{\text{cfg}} \left[\varepsilon_\theta(x_j, o_{\text{cond}}) - \varepsilon_\theta(x_j, \varnothing)\right], \quad \lambda_{\text{cfg}} = 1.5.$$

The DDIM step is then:

$$x_{j-1} = \sqrt{\bar{\alpha}_{j-1}}\,\hat{x}_0(x_j; \varepsilon_{\text{cfg}}) + \sqrt{1-\bar{\alpha}_{j-1}}\,\varepsilon_{\text{cfg}}.$$

### 2.3 Safety guidance term: cost on the predicted clean trajectory

Following Lan-o3dp (Li et al.) and MPD (Carvalho et al.), we define the collision cost on the **clean delta-action trajectory cumsum'd to world-frame waypoints**:

Step 1. Recover predicted clean delta-trajectory: $\hat{x}_0 \in \mathbb{R}^{H \times 11}$. Split into $\hat{x}_0^{\text{dpos}} \in \mathbb{R}^{H \times 3}$ (the first three action channels) — the per-step world-frame position deltas.

Step 2. Cumsum to world positions: $\hat{p}_k = p_0 + \sum_{i=1}^{k} \hat{x}_0^{\text{dpos}}[i], k = 1, \ldots, H$. Note this is differentiable; PyTorch `torch.cumsum` supports backprop.

Step 3. Transform body-frame point cloud $P_{\text{body}} \in \mathbb{R}^{4096 \times 3}$ to world frame *once per plan* (it does not depend on the diffusion step): $P_{\text{world}} = R_{b \to w} P_{\text{body}} + t_{b \to w}$. Build a fixed KD-Tree on

$P_{\text{world}}$ for k-NN queries.

Step 4. For each predicted waypoint $\hat{p}_k$, find its $k$-nearest neighbors $\{q_{k,1}, \dots, q_{k,m}\}$ in $P_{\text{world}}$ (m=8). The KD-Tree query is non-differentiable, but **we only differentiate through the Euclidean distance to the returned neighbor set**, which keeps the gradient well-defined as long as the active neighbor set is locally stable. (Standard trick used in DCNet/PCN-style differentiable Chamfer losses.)

Step 5. Define the soft-min distance for waypoint $k$:

$$d_k(\hat{p}_k) = -\frac{1}{\beta} \log \sum_{l=1}^{m} \exp\bigl(-\beta \, \|\hat{p}_k - q_{k,l}\|_2\bigr), \quad \beta = 20$$

This is a smooth approximation of $\min_l \|\hat{p}_k - q_{k,l}\|$.

Step 6. Softplus-hinge per-waypoint penalty around safety threshold $d_{\text{safe}} = 0.35$ m:

$$c_k(\hat{p}_k) = \frac{1}{\beta} \log\bigl(1 + \exp(\beta \, (d_{\text{safe}} - d_k))\bigr)$$

This is differentiable everywhere, smoothly turns on as $d_k$ approaches $d_{\text{safe}}$ from above, and reduces to roughly $(d_{\text{safe}} - d_k)_+$ for $\beta$ large. ($\beta = 20$ means transition width ~0.05 m, which matches our 0.05 m extra margin beyond the 0.30 m EGO inflation.)

Step 7. Aggregate horizon-wise with **higher weights on early waypoints** (those that will actually be executed):

$$c(\hat{x}_0) = \sum_{k=1}^{H} w_k \, c_k(\hat{p}_k), \quad w_k = \exp(-k/\tau), \; \tau = 8$$

This addresses the **multi-step lookahead** concern: all 16 steps are evaluated, but the first ~8 dominate.

### 2.4 Injection into the DDIM step

We follow Lan-o3dp's choice of injecting guidance into the ε-prediction (their ablation: "Using sample prediction can not successfully avoid obstacles. … we use epsilon prediction in real-world."): (arXiv)

$$\boxed{\tilde{\varepsilon}_j = \varepsilon_{\text{cfg}}(x_j) + \sqrt{1 - \bar{\alpha}_j} \cdot s_{\text{safe}} \cdot \alpha_j \cdot \nabla_{x_j} c\bigl(\hat{x}_0(x_j; \varepsilon_{\text{cfg}})\bigr)}$$

Then run the DDIM step with $\tilde{\varepsilon}_j$ instead of $\varepsilon_{\text{cfg}}$.

Derivation justification: Recall $\hat{x}_0 = (x_j - \sqrt{1-\bar{\alpha}_j}\,\varepsilon)/\sqrt{\bar{\alpha}_j}$, so increasing $\varepsilon$ by $\Delta\varepsilon$ decreases $\hat{x}_0$ by $(\sqrt{1-\bar{\alpha}_j}/\sqrt{\bar{\alpha}_j})\Delta\varepsilon$ and consequently increases the cost by approximately $-(\sqrt{1-\bar{\alpha}_j}/\sqrt{\bar{\alpha}_j})\nabla c^\top \Delta\varepsilon$. To *decrease* cost, we therefore want $\Delta\varepsilon \propto +\nabla c$ (note the sign — Lan-o3dp also adds the gradient, not subtracts, when working in ε-space). The factor $\sqrt{1-\bar{\alpha}_j}$ converts an $x$-space gradient to an ε-space update of the right scale (matches Dhariwal & Nichol's $\Sigma_t \nabla \log p$ normalization in the score-function view).

### 2.5 Per-step scale schedule $\alpha_j$

We have 5 DDIM steps: $j \in \{10, 8, 6, 4, 2\}$ (and the final 0). Three candidate schedules:

| Schedule | Formula | Rationale |
|---|---|---|
| (a) Linear ramp-up | $\alpha_j = 1 - j/10$ | MPD style: "increasing likelihood relevancy when σ_t → 0" (tu-darmstadt) |
| (b) Cosine middle-peak | $\alpha_j = \cos^2\big(\pi(j-5)/10\big)$ | Avoid disturbing anchor (early) and avoid over-correction near $x_0$ (late) |
| (c) Constant | $\alpha_j = 1$ | Baseline, simplest |

**Recommendation: (b) cosine middle-peak** as default, with (a) as ablation. The reasoning:

- At $j = 10$ (the first denoising step after truncation), $x_j$ is still dominated by anchor + small noise; pushing too hard here would overwrite the anchor's coarse direction choice (anchor compatibility — see §F below).
- At $j = 2$ (penultimate), the trajectory is nearly clean; large gradients here cause "trajectory deformation visible in the rollout" (Lan-o3dp also reports collapse with too-strong end-stage guidance, hence their cleaning sample-vs-epsilon choice).
- Middle steps $j \in \{4, 6, 8\}$ are where the trajectory has enough structure that $\hat{x}_0$ is a meaningful estimate but anchor commitment is not yet hardened.

[实验决定] The choice between (a) and (b) — sweep on 100 sim episodes, pick the schedule with the best collision-rate at fixed BC success.

### 2.6 Gradient clipping

Per-waypoint L2-norm clip:

$$g_k \leftarrow g_k \cdot \min\big(1,\ g_{\max}/\|g_k\|_2\big), \quad g_{\max} = 1.0$$

This implements the "anchor preservation" principle: even when one waypoint is deep inside an obstacle, the gradient cannot push by more than $\sim 1$ unit per step, so the anchor's direction choice is preserved.

### 2.7 Gradient stability of the softplus + softmin chain

The cost function involves nested non-linearities — `softplus(β·(d_safe - softmin_β(||·||)))`. This raises legitimate concerns:

- **First-order gradient**: well-defined everywhere. $\nabla_p d_k = \sum_l w_l (p - q_l)/\|p - q_l\|$ where $w_l = \exp(-\beta\|p - q_l\|)/Z$ are the softmin weights — a smooth convex combination of unit vectors. $\nabla c_k = -\sigma(\beta(d_{\text{safe}} - d_k)) \cdot \nabla d_k$ where $\sigma$ is the sigmoid. Both factors are bounded.
- **Second-order risks** (relevant if we ever propagate through guidance for training, e.g. unrolled RL): the second derivative of softplus has a peak of $\beta/4$ at the inflection point, and the softmin's Jacobian also scales with $\beta$. With $\beta = 20$, the curvature can reach $\sim 100$ at active points. **Mitigation**:
  - For *inference-time* guidance (our setting), only first derivatives are needed → not a concern.
  - For *training-time* differentiable guidance loops (future RL phase), reduce $\beta$ to ~10 during training and ramp up to 20 at inference. [实验决定 if RL fine-tune is enabled.]
  - The L2-norm gradient clip in §2.6 also bounds first-order magnitude, indirectly limiting how much second-order error matters per step.
- **Near-zero distance singularity**: when $\|p - q_l\| \to 0$ (waypoint on a point), the $(p - q_l)/\|p - q_l\|$ unit vector is undefined. Mitigation: add an $\varepsilon = 10^{-6}$ floor in the distance norm:

  $$\|p - q_l\|_\varepsilon := \sqrt{\|p - q_l\|^2 + \varepsilon^2}$$

  This is standard and costs nothing.
- **Active-neighbor set discontinuity**: the KD-Tree's k-NN returns a discrete neighbor index set. When a waypoint moves and the (k+1)-th nearest becomes the k-th, the active set changes discontinuously. The softmin smooths this out as long as k \ge $ ~5 (we use $k=8 ) — the neighbors leaving the set have very low softmin weights $\exp(-\beta d)$ so the gradient jump is small. Empirically (per Lan-o3dp's stability with $k$ in this range), this is not a major issue.

---

## 3. 算法伪代码 (iDP3 `conditional_sample` 兼容)

```python
```

```python
def conditional_sample_with_safety(
    self,
    cond_data,           # (B, To, obs_dim) - state
    cond_mask,
    pcd_world,           # (B, 4096, 3) world-frame point cloud (pre-transformed)
    p0_world,            # (B, 3) current drone position in world
    anchor_init,         # (B, H, 11) anchor head output, used as t_start initializa
    R_b2w, t_b2w,        # body-to-world transform (used once, outside)
    # Phase ②.5 hyperparams
    s_safe=2.0,
    d_safe=0.35,
    beta=20.0,
    g_max=1.0,
    k_nn=8,
    schedule="cosine_mid",  # one of "cosine_mid", "linear_up", "const"
    cfg_lambda=1.5,
):
    # Build KD-Tree once per plan (NOT per diffusion step) — O(N log N) ~ 1-2 ms for
    kdtree = build_kdtree(pcd_world)              # batched: scipy.spatial.cKDTree p
    inference_steps = self.num_inference_steps    # 5
    j_start = self.t_start                        # 10

    # Phase ②.5 — truncated init from anchor (unchanged from existing Phase ②)
    noise = torch.randn_like(anchor_init)
    x_j = self.noise_scheduler.add_noise(anchor_init, noise, torch.tensor([j_start])

    # Inference loop: 5 DDIM steps from j=10 to j=0
    timesteps = self.noise_scheduler.timesteps[-inference_steps:]   # [10, 8, 6, 4,
    for step_idx, j in enumerate(timesteps):
        # ----- A. existing CFG forward (cond/uncond concatenated batch) -----
        x_j_in = torch.cat([x_j, x_j], dim=0)                       # (2B, H, 11)
        eps_uncond, eps_cond = self.eps_model(x_j_in, j, cond_data_concat).chunk(2)
        eps_cfg = eps_uncond + cfg_lambda * (eps_cond - eps_uncond)  # (B, H, 11)

        # ----- B. Phase ②.5 — safety guidance -----
        x_j_for_grad = x_j.detach().requires_grad_(True)
        # Recompute eps_cfg on x_j_for_grad to enable backprop (cheap: 1 forward, sa
        eps_uncond_g, eps_cond_g = self.eps_model(
            torch.cat([x_j_for_grad, x_j_for_grad]), j, cond_data_concat
        ).chunk(2)
        eps_cfg_g = eps_uncond_g + cfg_lambda * (eps_cond_g - eps_uncond_g)

        alpha_bar_j = self.noise_scheduler.alphas_cumprod[j]
        x0_hat = (x_j_for_grad - (1 - alpha_bar_j).sqrt() * eps_cfg_g) / alpha_bar_j

        # Cumsum first 3 channels -> world waypoints
```

```
        delta_pos = x0_hat[..., 0:3]                                    # (B, H, 3)
        waypoints = p0_world.unsqueeze(1) + torch.cumsum(delta_pos, dim=1)  # (B, H,

        # k-NN distances (KD-Tree query is no-grad, distance computed with grad)
        with torch.no_grad():
            _, idx = kdtree.query_batched(waypoints.reshape(-1, 3).cpu().numpy(), k=
        neighbors = pcd_world.gather(1, idx_to_tensor(idx))             # (B, H, k_nn,
        diffs = waypoints.unsqueeze(2) - neighbors                      # (B, H, k_nn,
        dists = (diffs.pow(2).sum(-1) + 1e-12).sqrt()                   # ε-floored no
        # soft-min (Eq. in §2.3 step 5)
        d_k = -(1.0/beta) * torch.logsumexp(-beta * dists, dim=-1)      # (B, H)
        # softplus-hinge cost (Eq. in §2.3 step 6)
        c_k = (1.0/beta) * torch.nn.functional.softplus(beta * (d_safe - d_k))  # (B
        # horizon weights w_k
        w_k = torch.exp(-torch.arange(H, device=x_j.device) / 8.0)
        c_total = (w_k * c_k).sum(dim=1).sum()

        # Gradient on x_j
        grad_x = torch.autograd.grad(c_total, x_j_for_grad, retain_graph=False)[0]

        # Per-waypoint L2 clip
        grad_norm = grad_x.norm(dim=-1, keepdim=True).clamp(min=1e-8)
        grad_x = grad_x * torch.clamp(g_max / grad_norm, max=1.0)

        # Schedule
        if schedule == "cosine_mid":
            alpha_j = float(np.cos(np.pi * (step_idx - 2) / 4.0) ** 2)
        elif schedule == "linear_up":
            alpha_j = (step_idx + 1) / inference_steps
        else:
            alpha_j = 1.0

        # Inject into ε (Eq. boxed in §2.4)
        eps_safe = eps_cfg + (1 - alpha_bar_j).sqrt() * s_safe * alpha_j * grad_x

        # ----- C. DDIM step with eps_safe -----
        x_j = self.noise_scheduler.step(eps_safe, j, x_j).prev_sample

    return x_j  # final clean action trajectory
```

Notes on the code:

- **The forward is run twice per step** (once for ε_cfg in the no-grad path used for the actual DDIM update, once with requires_grad for guidance gradient). This is the dominant overhead (see §6). [实验决定] Whether to fuse them into a single forward by always keeping the graph; benchmark on T4.

- **The KD-Tree is built once per plan** (not per step). This costs ~2 ms for 4096 points; 5×0 = 0 additional in the loop.
- `kdtree.query_batched` uses CPU FAISS or scipy `cKDTree` — fastest at this point count; ~3–6 ms for 16 × B queries with k=8.
- `gather` **with idx_to_tensor** is the standard differentiable index-then-distance pattern (gradient flows through `diffs.norm`).

---

### 4. 工程实现路径 (具体到改在哪)

Assuming the iDP3 repo layout (`Improved-3D-Diffusion-Policy/diffusion_policy_3d/policy/`):

1. `policy/idp3.py` (or your `phase1a_policy.py`):
   - Locate `conditional_sample()` method (it is the one calling `self.noise_scheduler.step(...)` in a for-loop).
   - Replace the body with `conditional_sample_with_safety()` from §3.
   - Add hyperparameters `s_safe, d_safe, beta, g_max, k_nn, schedule` to `__init__` and pass through from config.
2. **New file** `policy/safety_guide.py`:
   - `class SafetyGuidance`: encapsulates KD-Tree building, cost computation, gradient clipping.
   - Methods: `build_tree(pcd_world)`, `compute_cost(waypoints)`, `compute_gradient(x_j, eps_cfg, eps_model, j, cond)`.
   - Keep it self-contained so ablation flags can swap it for a no-op.
3. **Config file** `config/task/uav_obstacle_avoidance.yaml`:

```yaml
policy:
  safety_guidance:
    enabled: true
    s_safe: 2.0
    d_safe: 0.35
    beta: 20.0
    g_max: 1.0
    k_nn: 8
    schedule: "cosine_mid"
    horizon_decay_tau: 8.0
```

4. **Workspace runner** `workspace/uav_phase25.py`:

- Add evaluation hooks for collision-rate, success-rate, average clearance, average plan time (already exist in Phase ②).
- Add CLI flags to flip `safety_guidance.enabled` for the 4-way ablation in §7.

5. **Body-to-world transform** (called by the inference loop, once per call before `conditional_sample`):

```python
# in policy forward()
R_b2w = quaternion_to_rotmat(state[:, 6:10])   # (B, 3, 3)
t_b2w = state[:, 0:3]                          # (B, 3) = drone position
pcd_world = (R_b2w @ pcd_body.transpose(-1, -2)).transpose(-1, -2) + t_b2w.unsqueeze
p0_world = t_b2w
```

6. **Testing checklist**:
   - Unit: with `s_safe=0`, output identical to Phase ② baseline (sanity).
   - Unit: gradient flows through `cumsum` and `gather`-then-distance (compare `torch.autograd.gradcheck`).
   - Integration: 100-episode sim eval, collision-rate vs success-rate sweep over `s_safe ∈ {0.5, 1.0, 2.0, 4.0}`.

---

## 5. 超参起步推荐 (具体数字)

| Hyperparameter | Symbol | Recommended start | Sweep range | Justification |
|---|---|---|---|---|
| Safety threshold | $d_{\text{safe}}$ | **0.35 m** | 0.25–0.50 | EGO B-spline inflation = 0.30 m + 0.05 m margin |
| Guidance strength | $s_{\text{safe}}$ | **2.0** | 0.5–5.0 | Start low, increase until collision drops without success drop. CGD's $\alpha_{\text{obst}} = 1.0$ is in their normalized space; ours has additional softplus → factor of ~2 |
| Softplus sharpness | $\beta$ | **20.0** | 10–50 | Transition width $\approx 1/\beta = 0.05$ m matches our extra margin |
| Per-waypoint grad clip | $g_{\max}$ | **1.0** | 0.5–2.0 | Empirical default for inference-time guidance in robotics; we did not find a verbatim "clip = X" in MPD/Lan-o3dp/DiffusionDrive PDFs — MPD |

| Hyperparameter | Symbol | Recommended start | Sweep range | Justification |
|---|---|---|---|---|
| | | | | instead controls magnitude via $\lambda_i / \sigma_t$ schedule |
| k-NN neighbors | $k_{\text{NN}}$ | **8** | 4–16 | Smooths soft-min over local point density without much cost |
| Horizon weight decay | $\tau$ | **8** | 4–16 | Emphasizes first half of horizon (executed steps) |
| Schedule type | — | **"cosine_mid"** | {const, linear_up, cosine_mid} | See §2.5 |
| Distance-norm floor | $\varepsilon$ | $10^{-6}$ | fixed | Prevents singular gradient at $\|p - q\| = 0$ (§2.7) |
| CFG λ (unchanged) | $\lambda_{\text{cfg}}$ | 1.5 | — | Keep iDP3 default; do **not** retune jointly with safety |
| t_start (unchanged) | $j_{\text{start}}$ | 10 | — | Keep truncated diffusion |

[UNCERTAIN] $s_{\text{safe}}$: Lan-o3dp and MPD do not publish a single scalar; their $\lambda_i$ per-cost weights vary across tasks. The range 0.5–5.0 is a reasonable bracket given our softplus normalization, but the optimum almost certainly depends on training-data statistics. Sweep is mandatory.

[实验决定] Schedule choice (cosine_mid vs linear_up) — sweep on a held-out 100-episode sim set.

---

## 6. 预期推理时间 overhead

Per DDIM step extra cost analysis (T4, batch_size effective B=1 for inference, CFG concat doubles to 2):

| Operation | Time | Notes |
|---|---|---|
| Extra ε forward (for grad) | ~10–15 ms | Same cost as the existing ε forward; CFG concat already at 2B |
| Compute $\hat{x}_0$ + cumsum | <0.5 ms | 16×3 ops |
| KD-Tree query (16 × k=8 per item) | ~3–5 ms | scipy cKDTree on CPU; can move to GPU FAISS if hot |
| `gather` + distance + softmin + softplus | ~1 ms | Pure tensor ops, 16×8×3 |
| `torch.autograd.grad` backprop | ~8–12 ms | Backprop through one U-Net forward; this is the second-largest cost |
| Gradient norm clip + ε update | <0.5 ms | Trivial |
| **Per-step total** | **~25–35 ms** | |
| **5-step total** | **~125–175 ms** | |
| KD-Tree build (once per plan) | ~2 ms | One-time |

**Plan-total overhead: ~130–180 ms**, on top of the current ~2.5 s/plan baseline → projected total ~2.6–2.7 s, well within the 3 s budget.

Optimization knobs if we exceed budget:

- **(a) Apply guidance only on middle 3 of 5 steps** ($j \in \{6, 4, 2\}$); saves 40% of guidance overhead. This is also more consistent with the cosine_mid schedule (which already has ~0 weight at endpoints). SafeDiffuser's RES-Diffuser-L10 result (0.011s vs 0.106s when restricting to last 10 of 256 steps) shows this is the standard speedup.
- **(b) Skip the extra forward**: keep the gradient graph on the original ε_cfg forward. Risk: doubles peak memory; need to verify on 24 GB T4. [实验决定]
- **(c) Replace scipy cKDTree with GPU FAISS** (preinit on GPU); ~3 ms saving but more deployment complexity.

[UNCERTAIN] Whether the "extra forward" is necessary. If iDP3's `conditional_sample` already retains the graph (it typically does not in `eval()` mode), we can re-enable graph retention only during guidance computation. Benchmark on T4 before committing.

---

## 7. 论文 ablation 设计

Four-way table on the same 200-episode held-out sim set + 30-episode real-world Nanchang campus set:

| ID | Method | Phase ② collision-aux loss | Phase ②.5 safety guidance | RL fine-tune | Expected coll. rate | Expected success rate |
|---|---|---|---|---|---|---|
| A | BC only | ✗ | ✗ | ✗ | High (~30–50%) | Baseline |
| B | BC + aux | ✓ | ✗ | ✗ | Mid (~10–20%) | ≈ A |
| C | **BC + aux + guidance** | ✓ | ✓ | ✗ | **Low (<5%)** | ≈ A |
| D | C + RL fine-tune | ✓ | ✓ | ✓ | Lowest (~0–2%) | > A |

Additional micro-ablations within row C:

- C1: schedule = const
- C2: schedule = linear_up
- C3: schedule = cosine_mid (recommended)
- C4: guidance on x_{t-1} instead of ε (the "MPD" injection point) — predicted to be slightly worse based on Lan-o3dp's epsilon-prediction result
- C5: $s_{\text{safe}} \in \{0.5, 1.0, 2.0, 4.0\}$
- C6: KD-Tree k $\in \{4, 8, 16\}$
- C7: $d_{\text{safe}} \in \{0.25, 0.35, 0.50\}$
- C8: guidance on all 5 steps vs middle 3 of 5 (speed/quality trade-off)

Metrics to report: collision rate, success rate (reach goal within time), average min-clearance, average plan time, qualitative anchor-direction-preservation rate (does the executed trajectory follow the anchor's chosen side?).

For the thesis, the headline plot should be **Pareto front of collision-rate vs success-rate** swept over $s_{\text{safe}}$, with the four ablation rows marked as points.

---

## 8. Outstanding open questions

1. **[UNCERTAIN] Anchor override frequency**: How often does strong guidance ($s_{\text{safe}} \geq 4$)

flip the anchor's direction choice (left → right) in the final trajectory? Measure by computing cosine similarity between anchor's coarse direction and final waypoint direction. If >10% flip rate, gradient clip is too loose.

2. **[实验决定] Sample-prediction vs ε-prediction guidance**: Lan-o3dp explicitly chose ε-prediction with the same cost (their words: "the guidance is almost ineffective if each step predicts the initial sample $A_0$ directly"). We follow them, but it is worth a one-shot sanity check: with $s_{\text{safe}}$ multiplied by $\sqrt{\bar{\alpha}_j/(1-\bar{\alpha}_j)}$, guide on $\hat{x}_0$ directly. If results differ markedly, document. (arXiv)
3. **[UNCERTAIN] Body-frame point cloud vs world-frame transform stability**: Our point cloud is captured in body frame, transformed once at plan-start. During the 5 denoising steps (~150 ms), the drone has moved (at 5 m/s, ~0.75 m). The KD-Tree built at plan-start becomes stale by the end of the plan. Mitigation: re-build per-step? [实验决定: the 0.75 m drift is within the 0.35 m d_safe + 0.30 m EGO inflation safety margin for typical obstacles, but at high speed in narrow gaps this could matter.]
4. **[UNCERTAIN] Multi-modal obstacle topology and local traps**: SafeDiffuser explicitly addresses "local trap problems" with ReS and TVS variants. Our k-NN soft-min has no such guarantee — when an anchor points the drone straight at a wall, the softplus-hinge gradient can push laterally arbitrarily, potentially against the wall normal direction. SafeDiffuser's solution (TVS with time-varying $\sigma_k(j)$) is incompatible with our 5-step window. Mitigation: rely on the anchor head to provide a feasible direction, and limit guidance to small corrections (the L2 clip). If local traps observed, fall back to per-step random restart of the anchor. (OpenReview)
5. **[UNCERTAIN] Interaction with the 0.3 m EGO B-spline inflation in training data**: Training demos already keep ⩾ 0.3 m. The diffusion policy has effectively learned the 0.3 m offset. If we set $d_{\text{safe}} = 0.35$ m, the guidance will mostly be inactive on in-distribution scenes and only activate on OOD or anchor-fail cases — which is exactly the right behavior. But if training data has rare close-pass episodes, the policy might have learned the close-pass mode and guidance will fight it. Solution: monitor $\mathbb{E}_j[c]$ on a held-out training set; should be near-zero if the policy is well-behaved in-distribution.
6. **[实验决定] Whether to add a velocity-component guidance**: We currently guide only the position-delta channels (action dim 0:3). The other 8 channels (dvel, acc, dyaw, yaw_rate) are *not* directly safety-relevant but influence dynamic feasibility. Adding a soft acceleration/jerk hinge (similar to CGD's dynamic-feasibility QP, but as a softplus penalty) is a clean extension.
7. **[UNCERTAIN] CFG λ interaction**: We assume $\lambda_{\text{cfg}} = 1.5$ unchanged. But CGD's "block coordinate descent" framing implies guidance interacts with the conditioning. If guidance dominates, the effective CFG is "drowned out". A safety check: confirm that conditional-vs-unconditional KL divergence on the final trajectories is unchanged (within 10%) between Phase ② and Phase ②.5.
8. **Hard-constraint fallback**: Phase ②.5 is soft-constraint by MC-1. If a downstream phase needs hard guarantees (Phase ③?), the natural extension is to add a **post-hoc velocity-limit + emergency-stop logic at the MPC tracker level** (not at the policy level — that preserves

MC-1) while keeping the diffusion policy itself soft-constrained. This is out of scope for ②.5 but should be noted.

---

## 9. Design-decision details (A–H, with 2–3 candidates + trade-off per question)

### A. Guidance math form

**Candidate A1: $\nabla \log p(\text{safe}|x_t)$ classifier-guidance on $\hat{x}_0$ (Lan-o3dp / DPS pattern).** Compute cost on Tweedie estimate, backprop through ε-model once. *Pro*: trivial integration; no extra learned classifier; works with point clouds via any differentiable cost; mathematically equivalent to score-based conditional sampling. *Con*: soft constraint only (no certificate); guidance can be drowned out by CFG at large $s_{\text{safe}}$.

**Candidate A2: CBF gradient (SafeDiffuser RoS).** Define a CBF $b(x) = d(x) - d_{\text{safe}}$, enforce $\dot{b} + \alpha(b) \geq 0$ via QP at each step. *Pro*: finite-time diffusion-invariance theorem provides safety certificate :antCitation[]{citations="6bc81f33-399f-439d-8c6b-c5833fa29d13" injected="space"} (within diffusion time, not necessarily robot time). *Con*: per-step QP is expensive — SafeDiffuser Table 2 shows 0.106 s/step in full setting; QP must be re-derived for 16-waypoint horizon (16 constraints) and is non-trivial on T4 without OptNet-style batched solver; "local trap" issues require ReS/TVS variants that are incompatible with the 5-step window (TVS needs σ_k(j) defined over many steps).

**Candidate A3: Repulsive potential field (CGD-style).** Add explicit $\alpha_{\text{obst}}/\|p - q\|$ push to each waypoint. *Pro*: closed-form, no autograd needed. *Con*: requires explicit obstacle centers (we have a point cloud, not centers); CGD's $1/\|p - q\|$ diverges at contact and needs ad-hoc capping; the gradient direction is purely radial-from-nearest-point, ignoring local surface topology.

**→ Recommendation: A1.** A1 is the only candidate that handles point-cloud-only obstacles natively. The CBF certificate of A2 is illusory in our setting anyway (we only have 5 denoising steps and a non-smooth cost), and A3 doesn't fit our representation.

### B. Differentiable obstacle distance proxy

**Candidate B1: k-NN soft-min (LogSumExp).** $d_k = -(1/\beta) \log \sum_l \exp(-\beta \|p - q_l\|)$ with $k = 8$ neighbors from KD-Tree. *Pro*: O(H·k) per query (~1 ms forward + ~3-5 ms KD-Tree); smooth gradient; matches Phase ② collision-aux loss for consistency. *Con*: depends on point cloud density (sparse regions give a poor estimate); no signed-distance information (waypoint inside the obstacle gets the same gradient direction as outside).

**Candidate B2: Online voxel SDF (e.g., voxblox-style).** Build a TSDF from the point cloud, query trilinearly. *Pro*: true SDF (signed distance) gives correct gradient even inside obstacles. *Con*: build cost: ~30+ ms for a 1m³ voxel grid at 0.05 m resolution; memory: ~100 MB; integration with autograd needs custom CUDA. Overkill for 5 denoising steps.

**Candidate B3: Gaussian KDE.** $d \approx -(1/\beta)\log\sum_l \exp(-\beta\|p-q_l\|^2)$ over *all* 4096 points (no k-NN). *Pro*: globally smooth; no neighbor-set discontinuity. *Con*: O(H·N) per query — 16 × 4096 = 65k ops per waypoint set; ~10-15 ms on T4. Heavier than B1 with no clear quality gain at our point cloud density.

**→ Recommendation: B1.** Fast, fits Phase ② consistency, gradient quality is adequate when waypoint stays outside obstacles (which is the normal case — guidance just keeps it there).

### C. Guidance scale schedule $\alpha_j$

**Candidate C1: Constant.** $\alpha_j = 1$ for all 5 steps. *Pro*: simplest. *Con*: applies same strength regardless of denoising stage; risks fighting anchor at step 0.

**Candidate C2: Linear ramp-up (MPD style).** $\alpha_j = (1 - j/10)$. Maps to (0.0, 0.2, 0.4, 0.6, 0.8) over j=10,8,6,4,2. *Pro*: matches MPD's verbatim observation "increasing likelihood relevancy when σ_t → 0"; weak when anchor is still being respected; strong when trajectory must be nearly correct. *Con*: very strong at the last step (which Lan-o3dp warns against in sample-prediction mode); leaves obvious collisions uncorrected in early steps.

**Candidate C3: Cosine middle-peak.** $\alpha_j = \cos^2(\pi(j-5)/10)$. Maps to (0.0, 0.65, 1.0, 0.65, 0.0) over j=10,8,6,4,2. *Pro*: protects anchor (α=0 at j=10) AND protects late-stage fine structure (α=0 at j=2); middle steps do the heavy lifting. *Con*: more hyperparameters implicit in the shape; no published precedent in robotics-diffusion literature (this is our recommendation, not a citation).

**→ Recommendation: C3** with C2 as a fallback ablation row. Literature is split between "early big" (image guidance scale annealing by Kynkäänniemi et al.) and "late big" (MPD's $\lambda_i/\sigma_t$); for a *short* truncated window in planning, splitting the difference is the safer first guess. [实验决定 final pick.]

### D. CFG compatibility — where to inject

**Candidate D1: Inject on $\varepsilon_{\text{cfg}}$ (recommended).** $\tilde{\varepsilon} = \varepsilon_{\text{cfg}} + \sqrt{1-\bar{\alpha}_j}\cdot s \cdot \alpha_j \cdot \nabla_x c$. *Pro*: respects CFG as the prior (CFG combines first, safety modulates after); does not require modifying iDP3's existing CFG concat; mathematically clean. *Con*: small risk of double-counting if both CFG and safety push the same direction.

**Candidate D2: Inject on $x_{t-1}$ directly (MPD style).** Compute new $x_{t-1}^{\text{cfg}}$ via DDIM, then $x_{t-1} = x_{t-1}^{\text{cfg}} + g$. *Pro*: simpler to reason about geometrically. *Con*: equivalent to D1 up to a $\sqrt{1-\bar{\alpha}_j}/\sqrt{\bar{\alpha}_{j-1}}$ rescaling (Dhariwal-Nichol Eq. 1); but harder to integrate with the noise_scheduler.step() API.

**Candidate D3: Re-combine CFG with safety as an additional "condition" channel.** Treat safety as a third condition and do a three-way CFG ε_safe = ε_uncond + λ_cond·(ε_cond - ε_uncond) + λ_safe·(ε_safe_cond - ε_uncond). *Pro*: principled. *Con*: requires training a third "safety-conditioned" network — violates MC-2 (no re-training).

**→ Recommendation: D1.** Keep $\lambda_{\text{cfg}} = 1.5$ unchanged; tune only $s_{\text{safe}}$.

**E. Truncated diffusion compatibility**

**Candidate E1: Apply guidance on all 5 steps.** *Pro*: maximum collision suppression. *Con*: ~150 ms overhead; may disturb anchor at step 0.

**Candidate E2: Apply only on middle 3 of 5 steps (j ∈ {6, 4, 2}).** *Pro*: 40% saving; aligns with cosine_mid schedule's near-zero weight at endpoints; precedent in SafeDiffuser RES-Diffuser-L10 (10/256 step restriction). *Con*: collisions visible in first/last step are uncorrected.

**Candidate E3: Apply only on first 3 of 5 steps.** Mirror of E2. *Pro*: matches "fix problems early" intuition. *Con*: contradicts MPD's evidence that guidance should grow as σ_t → 0; not recommended.

**→ Recommendation: E1 for paper-grade reporting (full ablation), E2 as default for deployment** if real-time budget is tight.

**F. Anchor head compatibility**

**Candidate F1: Hard separation — guidance disabled at j=j_start, gradually enabled.** Use cosine_mid schedule with α=0 at step 0. *Pro*: anchor's direction choice fully preserved at initialization. *Con*: needs careful schedule tuning.

**Candidate F2: L2 gradient norm clip (recommended in addition to F1).** Cap per-waypoint gradient magnitude. *Pro*: bounds the worst-case anchor override even at strong $s_{\text{safe}}$. *Con*: when an anchor really is infeasible (e.g., points the drone into a wall), the clipped gradient may be too weak to correct.

**Candidate F3: Anchor-conditioned guidance scale.** Compute the anchor's chosen "direction class" and reduce $s_{\text{safe}}$ projection along that direction. *Pro*: targeted preservation. *Con*: complex; requires defining "direction class"; not common in literature.

**→ Recommendation: F1 + F2 together.** F3 is a stretch goal for a follow-up paper.

**G. Hard vs soft constraints**

**Candidate G1: Pure soft (recommended given MC-1).** Softplus-hinge cost, no enforcement guarantee. *Pro*: differentiable, integrable into diffusion, no post-hoc step. *Con*: no zero-violation guarantee.

**Candidate G2: Augmented Lagrangian / increasing $\beta$.** As denoising progresses, increase $\beta$ from 10 to 50 so the softplus becomes nearly hinge-shaped at j=2. *Pro*: approaches hard-constraint behavior at the end; still MC-1-compliant (no post-hoc projection). *Con*: gradient curvature explodes at high $\beta$ (see §2.7).

**Candidate G3: SafeDiffuser-style CBF-QP.** Per-step QP enforcement. *Pro*: certificate. *Con*: per-step QP overhead; needs OptNet differentiable QP layer; conflicts with cumsum + softmin chain (the constraint Jacobian is non-trivial to build).

**→ Recommendation: G1 with G2 as an ablation row (anneal $\beta$ from 10→50 across 5 steps).** G2 is the safest "soft → hard approximation" that doesn't violate MC-1.

### H. Step-wise delta action + cumsum compatibility

**Candidate H1: Cumsum inside the autograd graph (recommended).** Compute $\hat{p}_k = p_0 + \text{cumsum}(\hat{x}_0^{\text{dpos}})$ inside the gradient computation. *Pro*: PyTorch `torch.cumsum` is natively differentiable; the backprop simply distributes the gradient backward (every delta sees the sum of gradients from waypoints downstream of it). *Con*: gradient on delta $i$ accumulates contributions from all waypoints $\geq i$, so deltas near the start of the horizon get larger gradients than those at the end — this could bias the policy to over-correct early. Mitigation: the horizon weights $w_k = \exp(-k/\tau)$ counteract this by giving early waypoints more cost weight, which keeps the *per-delta* gradient roughly uniform.

**Candidate H2: Per-waypoint independent gradient (no cumsum graph).** Treat each waypoint as an independent point; gradient on delta $i$ comes only from waypoint $i$, not the downstream chain. *Pro*: simpler, gradient is "local". *Con*: dynamically incoherent — moving waypoint 5 requires also moving deltas 1–5; if guidance only modifies delta 5, the rest of the trajectory still drifts into the obstacle.

**Candidate H3: Reparameterize policy output as absolute world positions.** Bypass the delta representation entirely. *Pro*: simplest gradient flow. *Con*: requires re-training the diffusion policy with a different action head, violating MC-2 (frozen training data with delta-form actions). Hard veto.

**→ Recommendation: H1.** Cumsum's natural gradient distribution combined with horizon weights ($\tau = 8$) handles the bias issue cleanly. The autograd graph adds <0.5 ms — negligible.

---

## 10. Challenges and pitfalls — explicit treatment (from user's §3 list)

| Challenge | Mitigation in this design | Section |
|---|---|---|
| Inference time budget (5 steps × guidance ≤ 100 ms ideal) | Per-step ~25-35 ms × 5 = ~150 ms (slightly over the 100 ms goal but within the 500 ms slack of the 3 s budget); aggressive option E2 reduces to ~90 ms | §6 |
| Gradient stability — softplus + softmin 二阶导 | First-order is bounded; second-order curvature peaks at $\beta/4 = 5$ for softplus and ~$\beta = 20$ for softmin Jacobian; not exercised at inference (only first-order needed); L2-clip bounds first-order; distance-norm floor $\varepsilon = 10^{-6}$ prevents zero-distance singularity | §2.6, §2.7 |
| Point cloud body frame → world transform | Transform once per plan (one matrix multiply ~0.5 ms), build KD-Tree in world frame, all subsequent queries are in world | §3 code + §4 step 5 |

| Challenge | Mitigation in this design | Section |
|---|---|---|
| Multi-step lookahead — all 16 vs end-only? | All 16 evaluated; horizon weights $w_k = \exp(-k/\tau)$ with $\tau = 8$ emphasize the first ~half (executed steps), but the tail still contributes a small signal so the policy doesn't generate "snake into obstacle at step 16" | §2.3 step 7 |

## Caveats and source-quality notes

- **Lan-o3dp's exact $Q^*$ and gradient-scale values**: confirmed via Moonlight's literature review of arxiv.org/pdf/2407.00451 v3 but **the verbatim equation and numerical constant were not retrievable** in this research session due to arxiv rate limits. The recommended $d_{\text{safe}} = 0.35$ m and $s_{\text{safe}} = 2.0$ values are calibrated to our specific training-data inflation (0.30 m EGO) rather than copied from Lan-o3dp; the Lan-o3dp numbers when checked may motivate adjustment.
- **MPD's $\lambda_i$ values** are not enumerated in the IROS paper main body; they appear in the released codebase. The schedule recommendation ("increasing likelihood relevancy when σ_t → 0") is verbatim from §III-D, but the absolute $\tilde{\lambda}_i$ magnitudes require re-tuning for our cost function.
- **DiffusionDrive's truncation** uses ratio 50/1000 (5%) and 2 inference steps; our setup is 10/50 (20%) and 5 steps. The shapes differ, but the principle (truncation + anchor) transfers cleanly and DiffusionDrive's Table 4 ablation suggests guidance over a small number of steps is sufficient when anchors carry the structural prior. (arxiv)
- **CGD's $\alpha_{\text{obst}} = 1.0$** is in their *normalized B-spline control-point space*, not in physical meters. Our $s_{\text{safe}} = 2.0$ is not directly comparable but is in a similar relative magnitude (multiplier on a softplus-normalized hinge).
- **SafeDiffuser's "0.011 s for last-10 step restriction"** is verbatim from Table 2 of the ICLR 2025 proceedings PDF and motivates our "guidance only in middle 3 of 5 steps" optimization knob in §6.
- **PC-Diffuser (Ku & Lyu)** is a recent preprint (2603.10330, late 2025); their reported 10.29% residual collision rate is a useful target but on a different domain (nuPlan, urban driving). Our domain (UAV, point-cloud obstacles) has different baselines; we should not directly transfer their numbers, only the comparative methodology.
- **CAPE (Yang et al., arXiv:2511.22773)** is a 2025-11 preprint, not yet peer-reviewed; cited for framing language only.
- **Cosine-middle-peak schedule (§2.5 candidate C3)** is *our* recommendation, not citing a specific published precedent — published planning-diffusion literature either uses constant or σ_t-rescaled schedules. The cosine shape is motivated by the dual constraints (preserve anchor at step 0, preserve fine structure at step 2) but should be ablated against constant and linear-ramp in row C of §7.

- **Subagent budget used (1/1)**: focused on Lan-o3dp / MPD / DiffusionDrive specific numbers; arxiv rate-limited the Lan-o3dp v3 PDF fetch, which is the main residual uncertainty.

---

## Completion table

| Spec item | Status | Section |
|---|---|---|
| E1 Lit review (5–8 papers, verbatim) | ✓ | §1 (8 papers, all with verbatim quotes + author lists) |
| E2 Math derivation | ✓ | §2 (Tweedie $\hat{x}_0$ chain, ε-injection boxed equation, schedule, clip, stability) |
| E3 Pseudocode iDP3-compatible | ✓ | §3 |
| E4 Engineering path | ✓ | §4 (files, configs, transform code) |
| E5 Hyperparameter starting values | ✓ | §5 (table with specific numbers) |
| E6 Overhead estimate | ✓ | §6 (~130–180 ms breakdown) |
| E7 Ablation design 4-way | ✓ | §7 + 8 micro-ablations |
| E8 Open questions | ✓ | §8 (8 items, each [UNCERTAIN] or [实验决定] tagged) |
| E9 Design-Q A–H, 2–3 candidates + trade-offs each | ✓ | §9 (full breakdown per question) |
| E10 Verbatim equations | ✓ | §1 SafeDiffuser/CGD/MPD/Lan-o3dp/DiffusionDrive/Diffuser |
| §3 challenges checklist (inference budget, grad stability, body-frame, lookahead) | ✓ | §10 explicit table |
| I1 Point-cloud setting (no SDF) | ✓ | §2.3 k-NN soft-min, §9 row B |
| I2 Body→world frame | ✓ | §4 step 5, §10 row 3 |
| I3 CFG+truncated+anchor compatibility | ✓ | §2.4, §8 items 1,7, §9 rows D/E/F |
| I4 cumsum diff | ✓ | §2.3 step 2, §3 code, §9 row H |
| I5 T4 reality | ✓ | §6 |
| I6 [UNCERTAIN] / [实验决定] tags | ✓ | scattered + Caveats section |